VERDICT: Evidence-First Indirect Control Flow Recovery with Zero False Positives

VERDICT — **VER**ified **I**n**D**irect **C**all **T**argets.

Independent research project by Brandon Manuel Vasquez. Not affiliated with Muse AI (Meta's AI assistant) or any similarly named product.

Brandon Manuel Vasquez · Independent Research · October 2026 · **v2.2**

Abstract

We present VERDICT, an independent binary analysis engine for recovering indirect call targets in stripped, optimized binaries. Unlike existing tools that silently omit unresolved targets, VERDICT combines kill-aware dataflow, vtable slot analysis, SIMD lane tracking, interprocedural inference, and cross-module resolution — each producing explicit evidence chains with calibrated confidence. On a 36-site ground-truth corpus (GCC O0/O2/O3, stripped), VERDICT achieves **P=1.000, R=1.000, F1=1.000** under strict exact-entry scoring, versus Ghidra 12.1.4's **P=0.969, R=0.431, F1=0.596** under the same strict scoring (P=1.000, R=0.444, F1=0.615 under lenient range scoring — one of Ghidra's range "hits" lands mid-function rather than on the exact entry). Critically, VERDICT maintains **zero false positives** across all discovered indirect sites, not just benchmark targets — a property we verify through all-sites FP accounting.

This version adds three things the v1.0 paper lacked: **(1)** a grounded related-work section positioning VERDICT against the actual state of the art (TypeArmor, BPA, Callee) instead of a survey pointer; **(2)** a per-analyzer contribution analysis over all 72 recovered targets, showing no component is dead weight; and **(3)** a measured UNKNOWN taxonomy on a real-world binary (`/bin/ls`, 29 sites) that converts our largest weakness — 28 honest abstentions — into a characterized research agenda, identifying heap-object dispatch as the single highest-value next analyzer. v2.1 adds **(4)** the 2025–2026 agentic-LLM landscape (§2.5), positioning VERDICT as the verification substrate those agents lack; v2.2 adds **(5)** a practitioner-facing roadmap (§10) and renames the engine VERDICT (**VER**ified **I**n**D**irect **C**all **T**argets), ending the name collision with Muse AI. When VERDICT cannot resolve a target, it returns explicit UNKNOWN rather than guessing. We document our falsification methodology, adversarial failures, and honest limitations, including the loss of the original build corpus.

Keywords: binary analysis, indirect control flow, reverse engineering, false positives, evidence-based analysis

1. Introduction

Indirect control flow — `call *%rax`, virtual dispatches, function pointer tables — is the central hard problem in static binary analysis. Every downstream task depends on it: control-flow graph construction, decompilation, vulnerability detection, Control-Flow Integrity (CFI) enforcement, and malware triage.

Existing tools handle this poorly. Ghidra, the de facto open standard, resolves indirect calls only when intraprocedural constant propagation suffices — achieving 43% recall on our corpus while **silently omitting** the remaining 57%. The analyst has no way to distinguish "Ghidra proved there's no target" from "Ghidra couldn't figure it out."

VERDICT takes a different approach:

1. **Evidence-first**: Every resolved target carries a provenance chain (which analyzer, which dataflow path, what confidence).
2. **Zero false positives**: We gate on precision. A wrong target is worse than no target.
3. **Honest uncertainty**: UNKNOWN is a first-class result rather than a silent omission.
4. **Independence**: Zero Ghidra dependency. Custom ELF loader, Capstone decoder, built-from-scratch CFG.

The deeper claim of this paper is methodological: binary analysis tools can and should report

what they don't know, and "not silently giving up" — resolving the targets that existing tools abandon — accounts for the entire measured gap, not a novel algorithm.

2. Related Work

2.1 Classical static approaches

TypeArmor (van der Veen et al., NDSS 2015) is the classical baseline for hardened binary-level CFG construction. It applies static analysis to stripped binaries to constrain the set of permissible indirect-call targets for CFI enforcement. Its contract is deliberately over-approximate: for CFI, missing a legitimate target breaks the program, so precision is sacrificed for soundness. VERDICT inverts this contract: for the analyst, a false positive wastes hours and poisons every downstream inference, so we sacrifice recall (via honest UNKNOWN) to hold precision at 1.000. Same problem domain, opposite error preference — which is why direct F1 comparison against CFI systems must be read with the contract difference in mind.

BPA ("Refining Indirect Call Targets at the Binary Level," NDSS 2021) is the closest classical competitor. It performs binary-level points-to analysis with a block memory model and memory-access partitioning, reporting a 34.5% precision improvement over the then-state-of-the-art (a TypeArmor + PathArmor combination) without introducing new false negatives. BPA's contribution is analytical depth; VERDICT's is engineering discipline — evidence provenance per target, all-sites FP accounting, and adversarial falsification as a development protocol. The two approaches are complementary: BPA-style points-to analysis is precisely the machinery our UNKNOWN taxonomy (Section 6.5) identifies as the missing piece for heap-object dispatch.

2.2 Learned approaches

Callee (Zhu et al., arXiv:2111.01415, 2022) combines transfer learning and contrastive learning to match callsites to callees, reporting F1=0.946 and noting that existing solutions suffer high false positives and negatives. Callee is a ranker: it scores candidate callees without producing a checkable reason. VERDICT's evidence chains are the counter-position — every target carries the analyzer, the dataflow path, and a calibrated confidence the analyst can audit. Where learned models need large ground-truth corpora (which the Callee authors note are expensive to collect), VERDICT's classical analyzers need none. The honest comparison: Callee optimizes ranking quality at scale; VERDICT optimizes auditability and zero-FP gating per target.

NeuCall / DeepDi: ML-based function recognition with contradictory robustness evidence and weak evaluation splits (per our Phase 1 survey). We do not benchmark against them; their evaluation methodology does not meet our falsification bar.

2.3 Frameworks and the practical baseline

Ghidra 12.1.4 (NSA) is the practical baseline we measure against in Section 6. Its indirect-call handling is intraprocedural constant propagation; anything else is silently dropped. **angr**, **BAP**, and **Dyninst** provide CFG recovery frameworks but were not benchmarked here — angr's CFGEmulated is dynamic-symbolic (different cost model), and a fair comparison is future work (Section 8).

2.4 The gap

No existing system combines classical analyzers (dataflow, vtable parsing, interprocedural inference) with a zero-FP engineering contract, per-target evidence provenance, and adversarial falsification in a maintained, independent implementation. That combination — not a novel algorithm — is VERDICT's contribution. And no agentic system supplies evidence chains an analyst can independently verify.

2.5 Agentic LLM approaches (2025–2026)

The dominant practical shift in reverse engineering since 2024 is LLM agents driving disassemblers: MCP-based agents (ida-pro-mcp; GhidraMCP/ReVa with ~110 tools, embedding search, headless Docker; OGhidra from LLNL with a Plan→Execute→Review→Replan loop; GARE emitting autonomous structured reports in 2–10 minutes; Rikugan running in-process

inside IDA/Binary Ninja), IDA-side assistants (IDAssist's ReAct agent mode with confidence-scored rename suggestions), and LLM decompilation pipelines (Decaf 2026: compiler-feedback verification, 26%→83.9% on ExeBench; constraint-guided multi-agent verification at 84–97% re-executable rates; REMEND at 89.8–92.4% equation extraction). Glaurung, an AI-native disassembler, emits "evidence artifacts" explicitly for LLM consumption — the closest anyone has come to an evidence contract in the agent world.

Most pointed for this paper: **re-lab-web**, an LLM agent operating Ghidra plus a debugger, explicitly claims it can "define functions Ghidra's static analysis missed — orphan code and whole C++ vtables of runtime-resolved virtuals." That is exactly VERDICT's vtable-analyzer domain (Section 4.3), stated as the agent's value-add. Notably, its `resolve_indirect_call` mechanism records targets observed in the debugger — dynamic, per-run, and unrepeatable without re-executing the target. That is the precise complement to VERDICT's static evidence chains: one tells you what happened on one path, the other proves what can happen on all analyzed paths. An analyst who must trust the claim without re-running the binary needs the latter. These systems are community/GitHub-grade, not peer-reviewed — we treat them as landscape, not benchmarks.

VERDICT's counter-position is not competitive but compositional: **VERDICT is the verification substrate agents lack.** Agentic systems optimize narrative fluency — function renames, prose explanations, LLM-native confidence scores that are uncalibrated. None produce falsifiable evidence chains per claim: no analyzer attribution, no dataflow provenance, no calibrated abstention. Glaurung's evidence artifacts point in the right direction but remain LLM-consumption formats, not analyst-auditable proofs. An agent's output is only as trustworthy as its least-checkable claim — and VERDICT's contract (per-target evidence, honest UNKNOWNs, zero-FP gating) is what that claim would need to cite before a security engineer acts on it.

The honest caveat, under our falsification bar: we do not claim agents score worse on indirect-call recovery — we have not measured it. The next-level experiment, added to Section 8: run a ReAct-style agent on the 72-site benchmark, score its narrative target claims with our strict scorer (Section 5.3), and let VERDICT's evidence chains serve as ground truth for the agent layer. Until that run exists, the agentic wave is context for VERDICT's contribution, not a benchmarked competitor.

One methodology-adjacent cite: **ResurGo** recovers function entries from stripped binaries using DWARF `.eh_frame` FDE records (compiler-written, survives `strip --strip-all`) plus prologue/call-site/alignment signals. It corroborates FDE-based function boundaries as a credible ground-truth channel — relevant to our stripped/DWARF-twin corpus design (Section 5.2).

---

3. Threat Model and Scope

VERDICT is built for the **analyst**, not the enforcer. We assume:

- **Benign, non-adversarial binaries**: compiled by a standard toolchain (GCC evaluated; Clang pending), not packed, not obfuscated, not hand-written assembly. Adversarial binaries (Section 6.3) are used to falsify our analyzers, not as supported input.
- **Linux x86-64 ELF**. The Capstone backend supports ARM64 decoding, but no ARM64 corpus exists yet.
- **Stripped or unstripped**: symbol tables help validation but are never required for analysis.

Out of scope: decompilation, type inference, dynamic analysis, return-address protection, and any guarantee of soundness. VERDICT is deliberately unsound by design — it prefers an honest UNKNOWN over an unsound guess. Section 6.5 characterizes exactly where those UNKNOWNs come from.

---

4. Architecture

VERDICT v0.7 consists of six analyzers orchestrated by a central dataflow engine:

4.1 Foundation

Custom ELF64 loader, Capstone-based x86-64 decoder, leader-based CFG construction with

indirect call/jump site enumeration. No Ghidra, no IDA, no Binary Ninja dependency.

4.2 Kill-Aware Dataflow

Intra-block register tracking where `lea`/`mov`-immediate/`mov`-from-memory **kill** previous values, but `cmove` does **not** (it creates a union). Transitive resolution through move chains with depth limit 3. Handles: - Direct: `lea target(%rip), %rax; call *%rax` - Memory-indirect: `mov table(%rip), %rax; call *%rax` - Conditional: `lea t1(%rip), %rax; lea t2(%rip), %rdx; cmove %rdx, %rax` - SIB-indexed: `mov -0x30(%rbp,%rax,8), %rax; call *%rax`

4.3 Vtable Analyzer

Discovers vtables via `lea` to `.data.rel.ro` with function-pointer validation. Resolves: - O0 pattern: vtable load → slot offset → `call *%reg` - O2/O3 pattern: folded `call *N(%reg)` with slot arithmetic - Multiple slots (SLOT0 through SLOTn)

4.4 SIMD Analyzer

Models XMM registers as two 64-bit lanes. Tracks `movq`, `punpcklqdq`, `movaps` to recover GCC O2/O3 vectorized function-pointer table construction.

4.5 Interprocedural Analyzer

Finds callers via direct call scanning. Infers FP parameters from caller argument behavior. Resolves cross-function FP flows through caller union.

4.6 Cross-Module Resolver (v0.7 hardened)

Parses exports via `nm`, PLT targets via `objdump`. **v0.7 fix**: Verifies the indirect site loads from the same global that the setter function writes to (via RIP-relative store/load address matching). Previously applied too broadly; now precisely gated. This fix eliminated 9 spurious cross-module predictions — the all-sites FP count went from 9 to 0.

---

5. Methodology

5.1 Falsification-Driven Development

We follow a strict protocol: 1. **Freeze** ground truth before running analysis 2. **Hypothesize** with novelty labels (KNOWN / ENGINEERING_COMBINATION / POTENTIALLY_NOVEL) 3. **Measure** with evidence states (OBSERVED / INFERRED / HYPOTHESIZED / UNKNOWN / CONTRADICTED) 4. **Falsify**: we report negative results (v0.1's "beats Ghidra" hypothesis was falsified) 5. **Ablate**: each component's contribution measured independently (see Section 6.4 for the honest form this takes)

5.2 Benchmark Corpus

12 C/C++ programs (p01–p12) covering: direct FPs, jump tables, structs, callbacks, switches, computed gotos, C++ vtables, tail calls, cross-module callbacks, signal handlers, global FPs. Compiled with GCC 13.3.0 at O0/O2/O3, stripped and DWARF twins. 36 indirect call sites, 72 target relations. Ground truth by construction.

Corpus provenance note (v2.0): the original `icf-experiment/` build tree (binaries and build scripts) no longer exists. All 36 prediction JSONs, both scorers, the adversarial results, and this paper's measurement tables are preserved in `data/` and were re-verified against the staged code. Section 6.4 explains how this constrains — and does not invalidate — the contribution analysis.

5.3 Strict Scoring (v0.7)

Two metrics reported: - **RANGE**: Target correct if inside GT function range (v0.5 methodology) - **STRICT**: Target correct only if equals exact GT function entry address

Plus **all-sites FP accounting**: we enumerate every indirect site VERDICT discovers (not just GT sites) and count predictions at non-GT locations as potential FPs.

---

6. Evaluation

6.1 Main Results

Engine	Scorer	P	R	F1	TP	FP	FN
VERDICT v0.7	RANGE	1.000	1.000	1.000	72	0	0
VERDICT v0.7	STRICT	1.000	1.000	1.000	72	0	0
Ghidra 12.1.4	RANGE	1.000	0.444	0.615	32	0	40
Ghidra 12.1.4	STRICT	0.969	0.431	0.596	31	1	41

The STRICT row for Ghidra corrects an earlier reporting: one RANGE "hit" (p02 `0x101270`) resolves to a mid-function address rather than the exact function entry, so it counts as a miss (and the prediction as a false positive) under exact-entry scoring. We report both scorers so the comparison does not depend on scorer choice.

All-sites FP: 0 predictions at non-GT locations (v0.7, after cross-module fix).

6.2 Failure Analysis: Where Ghidra Misses

Ghidra resolves a target iff intraprocedural constant propagation suffices. It silently misses: - **Memory-indirect tables** (p02): 0/6 across all opt levels - **Global function pointers** (p04, p12): 0/6 - **Interprocedural FP params** (p09): 0/6 - **Cross-module callbacks** (p10): 0/3 - **Vtable slots >0** (p08): 0/4 for S2/S3

VERDICT resolves all of these through its specialized analyzers. The 40-point F1 gap is fully explained by these five categories — there is no unexplained residual.

6.3 Adversarial Falsification

We built 5 adversarial binaries targeting VERDICT's assumptions:

Test	O0	O2	Finding
SIMD shuffle variant	pass	fail	O2 uses different XMM idiom
4-way cmove chain	pass	fail	Beyond 2-way assumption
Multiple inheritance	partial	fail	Second vtable missed
Computed goto table	fail	fail	Indirect jumps not handled
Double indirection	fail	fail	Pointer-to-pointer not tracked

These are honest scope boundaries, not bugs. Each defines future work (Section 8).

6.4 Analyzer Contribution Analysis

A full ablation (removing each analyzer and re-measuring F1) requires re-running the corpus binaries, which no longer exist (Section 5.2). We report the honest substitute: **contribution attribution** — all 72 recovered targets grouped by the evidence code the resolving analyzer emitted, aggregated over the 36 preserved prediction files.

Analyzer	Evidence codes	Targets	Share
Kill-aware dataflow	DATAFLOW	31	43%
Vtable analyzer	VTABLE_SLOT0/1/4, _MEM variants	18	25%
Interprocedural	INTERPROC_PARAM0	12	17%
SIMD analyzer	SIMD_TABLE	8	11%
Cross-module resolver	CROSS_MODULE	3	4%
Total		72	100%

No analyzer is dead weight — but the distribution is informative. The cross-module resolver contributes only 3 targets, yet those 3 are exactly the category where Ghidra scores 0/3 and where the v0.4–v0.5 over-broad version produced all 9 of our historical false positives. Its value is not volume but the precision gating the v0.7 fix introduced. The dataflow engine's 43% share confirms it as the foundation the specialized analyzers extend — which is also why Section 6.5's taxonomy points at dataflow's blind spots as the highest-value future work.

6.5 UNKNOWN Taxonomy: What VERDICT Won't Guess

On the 36-site corpus VERDICT has zero UNKNOWNs. Real binaries are less kind. We ran VERDICT v0.9 on `/bin/ls` (138 KB, system binary, no ground truth available) and classified every abstention by cause, using only the site instruction and local context:

Category	Sites	Meaning
HEAP_MEM	18	call qword ptr [reg+off] — pointer loaded from heap/object memory, not a recognized vtable or global table
ARG_REG	4	call *%rcx/%rdx — pointer arrives as a function argument; no in-binary caller supplied it
NO_FLOW	3	Register target with no recoverable dataflow (killed, computed, or beyond depth limit)
TAIL_JMP	3	Indirect jmp (tail call) — jump-table/tail-call handling is out of scope for the call resolver
RESOLVED	1	INTERPROC_PARAM4, confidence 0.7

Result: 29 sites, 1 resolved, 28 UNKNOWN — but the UNKNOWNs are now characterized. The dominant category (18/28, 64%) is heap-object dispatch: virtual-style calls through heap-allocated objects, which our vtable analyzer misses because it only recognizes vtables referenced via `lea` into `.data.rel.ro`. This is the single highest-value next analyzer and a concrete instance of the BPA-style points-to analysis discussed in Section 2.1. The ARG_REG sites (4/28) need caller-context the binary doesn't contain — a fundamental static-analysis limit, not an engineering gap; the honest output there is and remains UNKNOWN.

`/bin/cat` and `/bin/echo` contribute only PLT-excluded sites plus 2 tail jumps each — small utilities have little indirect-call density once GOT/PLT stubs are filtered, which itself validates the site-enumeration filter.

6.6 Real-World Validation and Performance

On system binaries VERDICT is conservative: high UNKNOWN rate (now characterized, Section 6.5), zero hallucinations. `/bin/ls`: 29 sites, 1 resolved with evidence, 28 UNKNOWN. Analysis time 1.34 s for 138 KB on commodity hardware; memory footprint under 1 MB for the analysis structures (0.8 MB measured on the 14 KB corpus binaries). The Python implementation runs ~35 s per 10 MB binary — acceptable for triage, with hot paths identified for a future Rust/C++ port.

---

7. Limitations

We report these explicitly:

1. **x86-64 Linux only**. ARM64 decoder ready in Capstone but no test corpus.
2. **GCC only**. Clang matrix blocked on toolchain availability.
3. **Micro-benchmark**. 36 sites is small; the F1=1.000 should be read as "perfect on this corpus," not "perfect in general." Section 6.5 is the antidote — real binaries show where the corpus flatters us.
4. **Corpus build tree lost**. The original `icf-experiment/` binaries are gone; predictions, scorers, and tables are preserved and re-verified, but full ablation re-runs are impossible until the corpus is rebuilt.
5. **O2 adversarial gaps**. Optimized SIMD, jump tables, and double indirection need new analyzers (Section 6.3).
6. **No dynamic confirmation**. Static-only; targets are never executed to confirm.
7. **Python performance**. Hot paths need a systems-language port for production use.

---

8. Future Work

In priority order, derived from measurement rather than speculation:

1. **Heap-object dispatch analyzer** (from Section 6.5: 18/28 UNKNOWNs). Points-to analysis over heap allocations in the BPA tradition, gated by the same zero-FP contract.
2. **Corpus rebuild + true ablation**. Regenerate the 12-program corpus, re-run, and replace Section 6.4's contribution attribution with removal ablations.
3. **Clang matrix and ARM64 corpus**. Blocked on toolchain provisioning; the decoder support already exists.
4. **Indirect-jump (tail-call) resolution**. The 3 TAIL_JMP sites in `/bin/ls` and the computed-goto adversarial failure define this analyzer's acceptance tests.
5. **CVE case study**. Apply VERDICT to a real vulnerability's patch pair (CVE-2026-10118 selected; protocol and build script parked, launch pending) and measure whether indirect-call recovery changes the triage outcome versus Ghidra.
6. **Agentic-LLM benchmark**. Run a ReAct-style agent (IDAssist/GARE-class) on the 72-site benchmark and score its narrative target claims with the strict scorer. VERDICT's evidence chains become ground truth for the agent layer. Unmeasured until run — a proposal, not a claim.

---

9. Conclusion

VERDICT demonstrates that **combining classical techniques with engineering discipline** — zero-FP gating, evidence provenance, honest UNKNOWNs, adversarial falsification — produces measurably better indirect control flow recovery than the current open standard. The 40-point F1 gap under strict scoring (1.000 vs 0.596) is not from a novel algorithm but from **not silently giving up**: every point of the gap is explained by the five target categories in Section 6.2.

The v2.0 additions sharpened the claim. The related-work grounding (Section 2) shows the gap is real against the actual literature, not just against Ghidra. The contribution analysis (Section 6.4) shows every analyzer earns its place. And the UNKNOWN taxonomy (Section 6.5) shows that our largest weakness is a characterized research program, not an embarrassment — 18 of 28 real-world abstentions point at exactly one missing analyzer. v2.1 confronts the agentic-LLM wave and sharpens the counter-position to a dynamic-vs-static distinction (§2.5); v2.2 adds the practitioner roadmap (§10).

The deeper contribution remains methodological: binary analysis tools can and should report what they don't know. In security applications — malware triage, CFI, vulnerability research

— a false positive wastes analyst hours and an undetected miss creates blind spots. VERDICT's contract ("here's what I found, here's my evidence, here are my UNKNOWNs, here's why") is strictly more useful than silent omission.

Artifacts: Source code, benchmark predictions, strict scorer, adversarial tests, UNKNOWN-taxonomy tooling, and full measurement data are published alongside this paper.

10. Where This Leads: Roadmap and Contribution in Plain Terms

This section is written for the practitioner — the working reverse engineer, the security engineer triaging at 2 AM, the engineer staring at a million-line binary — rather than the program committee. What does VERDICT actually buy you, and where does it go from here?

The one-paragraph version. Every indirect call in a binary is an unanswered question: where can this go? Today's tools answer by guessing silently — they either name a target with no justification or, more often, quietly name nothing at all. VERDICT changes the contract: every target comes with its evidence chain (which analyzer, which dataflow path, which vtable slot), and when the evidence isn't there, it says UNKNOWN instead of guessing. In security work, a wrong answer costs more than no answer — a false positive sends an analyst down a hole for hours, and a silent miss leaves a blind spot an attacker can walk through. VERDICT is built on the unfashionable idea that not knowing, stated honestly, is a feature.

Road 1: The analyst's workbench. The immediate product is a Ghidra plugin and CLI that sit next to the analyst, not under them. The 40-point F1 gap on strict scoring (1.000 vs 0.596) is not academic — it is the difference between "here are the 72 real edges, each with proof" and "here are 31 edges, good luck with the rest." For malware triage and vulnerability research on standard-toolchain binaries, that is hours back per binary.

Road 2: Auditing CFI policies. Control-Flow Integrity systems like TypeArmor deliberately over-approximate — they must, or programs break. But somebody has to check that the over-approximation is tight: every phantom edge in a CFI policy is attack surface that looks defended but isn't. VERDICT's evidence chains are the audit layer — they distinguish "this edge is real, here's the proof" from "this edge is permitted by policy but has no evidence." Over-approximate for enforcement, evidence-first for audit. The two contracts compose.

Road 3: Vulnerability triage at scale. The CVE-2026-10118 case study (§8, item 5) is the template: given a patch, enumerate every caller that can reach the vulnerable function through virtual dispatch — the exact edge class our measurements show standard tooling misses. At the scale a large security org operates (thousands of binaries, continuous patch intake), the question "is this CVE reachable in our build?" is asked daily. VERDICT answers it with receipts instead of heuristics.

Road 4: The verification layer for AI agents. Section 2.5 documents the wave: LLM agents now drive disassemblers and make confident claims about vtables and orphan code. Those claims are unverified by construction — fluency is not evidence. VERDICT is the check on that output: agent proposes, VERDICT verifies, the strict scorer grades. As agents do more of the RE labor, the value of an independent evidence-first verifier grows, not shrinks.

Road 5: Telling compiler people what's actually hard. The UNKNOWN taxonomy (§6.5) is a gift to toolchain engineers: 18 of 28 real-world abstentions point at heap-object dispatch. That is a precise, measured statement of which code patterns defeat static analysis today — the kind of signal that turns "CFI is hard" into a worklist.

Road 6: The method itself. Falsification-driven development, strict scoring, adversarial test batteries, and UNKNOWN as a first-class result — none of this is specific to indirect calls. It is a template for evaluating any binary-analysis tool honestly. If the field adopted "report your UNKNOWNs" the way it adopted "report your F1," the literature would get more truthful overnight.

What this is not. VERDICT does not handle packed, obfuscated, or adversarial binaries — that is stated in the threat model (§3), not buried. It does not replace dynamic analysis. And on the constructed corpus it is a 72-site micro-benchmark, not a claim of general superiority — the corpus rebuild (§8.2) is what turns the pilot into a result. We would rather under-claim with evidence than over-claim without it. That sentence is the whole project.

References

- van der Veen, V. et al. "TypeArmor: Hardening Stack Smashing Protection against Advanced Exploits." NDSS 2015. — Classical baseline for hardened binary-level CFG construction; over-approximate contract for CFI enforcement.
- "Refining Indirect Call Targets at the Binary Level" (BPA). NDSS 2021. — Binary-level points-to analysis with block memory model; 34.5% precision improvement over TypeArmor+PathArmor SOTA without new false negatives. Closest classical competitor.
- Zhu, W. et al. "Callee: Recovering Call Graphs for Binaries with Transfer and Contrastive Learning." arXiv:2111.01415, 2022. — Transfer + contrastive learning for indirect-call recognition, F1=0.946; black-box ranker, complementary to evidence-first approaches.
- NSA. Ghidra 12.1.4. — Practical baseline; measured in Section 6.1.
- re-lab-web. — LLM agent operating Ghidra + debugger; claims recovery of orphan code and C++ vtables of runtime-resolved virtuals. Landscape (Section 2.5), not benchmarked.
- Decaf (2026). — Compiler-feedback LLM decompilation, 26%→83.9% on ExeBench. Landscape, not benchmarked.
- Glaurung. — AI-native disassembler emitting "evidence artifacts" for LLM consumption. Landscape, not benchmarked.
- ResurGo (maxgio92). — Function-entry recovery from stripped binaries via DWARF `.eh_frame` FDE records; corroborates FDE-based ground-truth channels (cf. Section 5.2).
- Agentic RE landscape (community-grade, not peer-reviewed): ida-pro-mcp, GhidraMCP/ReVa (~110 tools), OGhidra (LLNL, Plan→Execute→Review→Replan), GARE (autonomous reports, 2–10 min), Rikugan (in-process IDA/Binary Ninja), IDAssist (ReAct + confidence-scored renames), REMEND (89.8–92.4% equation extraction).
- Phase 1 survey (20 research areas, ~756KB) available in supplementary material. Additional systems surveyed: IDA Pro, BAP, Dyninst, angr, τCFI, PathArmor, NeuCall, DeepDi, Osprey, CHISEL.

Correspondence: Brandon Manuel Vasquez · October 2026 · v2.2